An Elastic Job Scheduler for HPC Applications on the Cloud

CANOPIE-HPC 2025

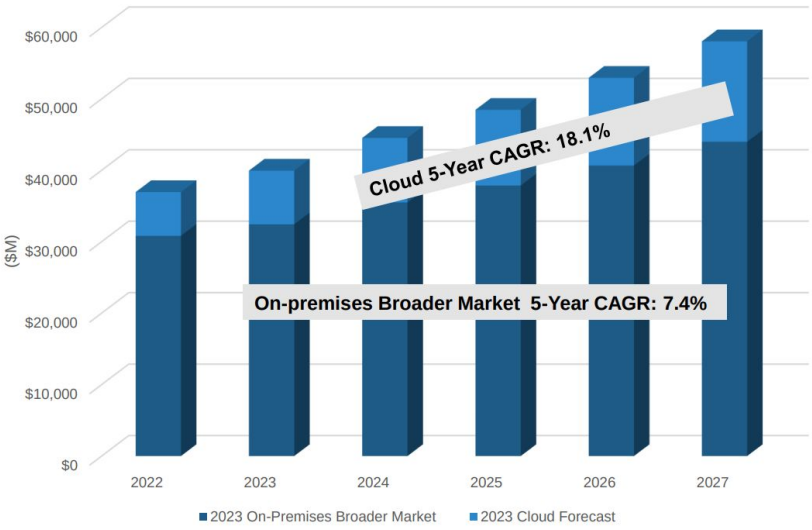
Aditya Bhosale, Kavitha Chandrasekhar, Laxmikant Kale
(University of Illinois Urbana-Champaign)

Sara Kokkila-Schumacher
(IBM Research)

Introduction

- Increasing availability and adoption of HPC cloud resources over the past few years

- The demand for HPC cloud resources is higher than ever with the increasing number of AI workloads running on cloud



Source: Hyperion Research, 2023 - https://www.ibm.com/downloads/cas/RGKYOOKB#:~:text=HPC%20in%20the%20cloud%20has,resources%20more%20warmly%20than%20others.

HPC on Cloud

- Kubernetes Operators for job scheduling and lifecycle management
- MPI Operator widely used for MPI applications

```
spec:
  slotsPerWorker: 1
  mpiReplicaSpecs:
    Launcher:
      replicas: 1
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-launcher
            command: <mpirun command>
    Worker:
      replicas: 4
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-worker
            resources:
              limits:
                cpu: 1
```

HPC on Cloud

- Kubernetes Operators for job scheduling and lifecycle management
- MPI Operator widely used for MPI applications

```
spec:
  slotsPerWorker: 1
  mpiReplicaSpecs:
    Launcher:
      replicas: 1
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-launcher
            command: <mpirun command>
    Worker:
      replicas: 4
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-worker
            resources:
              limits:
                cpu: 1
```

HPC on Cloud

- Kubernetes Operators for job scheduling and lifecycle management
- MPI Operator widely used for MPI applications
- Application runs with `1 + workerReplicas` MPI ranks

```
spec:
  slotsPerWorker: 1
  mpiReplicaSpecs:
    Launcher:
      replicas: 1
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-launcher
            command: <mpirun command>
    Worker:
      replicas: 4
      template:
        spec:
          containers:
          - image: <image>
            name: mpi-worker
            resources:
              limits:
                cpu: 1
```

Motivation

- The pay-as-you-go cost model of cloud makes it critical to use these resources efficiently

- Design goals for a job scheduler

  - Maximize cluster utilization
  - Minimize response times for high-priority jobs

Rigid Scheduling

- FCFS with user-defined priorities
- Backfilling to improve cluster utilization
- Experimental setup -

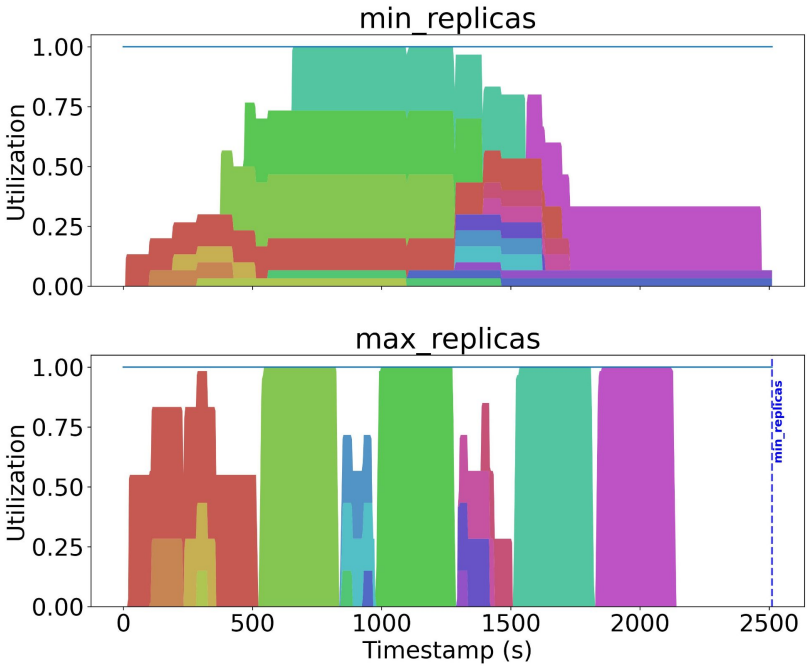
  Application: 2D Laplace equation solver with different grid sizes
  Metrics: Total time, cluster utilization, weighted mean response time

  Amazon EKS cluster with 4 c6g.4xlarge instances with 64 vCPUs total

  min_replicas - Batch of 16 jobs with different sizes and smaller number of replicas
  max_replicas - Same batch of jobs with 4 times the number of replicas

Rigid Scheduling

- min_replicas fails to saturate the cluster due to lower degree of parallelism
- Even though max_replicas has a high degree of parallelism, it leaves holes in utilization



	Total time (s)	Utilization	MRT (s)
min_replicas	2511	58.12%	162.87
max_replicas	2149	81.32%	147.51

Moldable Scheduling

- Jobs can start with any number of replicas between a min and max specified by the user
- Improve utilization by adapting job parallelism to the cluster state
- Measure weighted mean completion time since number of replicas is chosen by the scheduler

  Completion time = Time from job submission to job completion

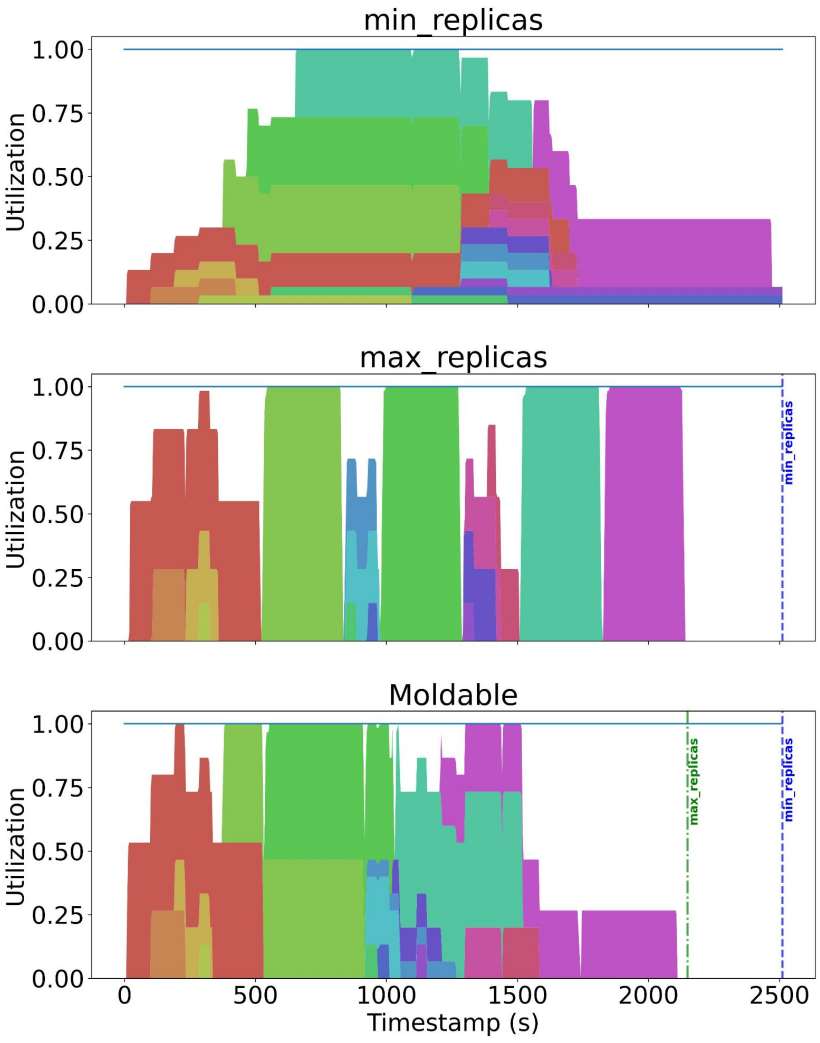
Moldable Scheduling

- We ran the same set of jobs with a moldable scheduling strategy

	Total time (s)	Utilization	MRT (s)	MCT (s)
min_replicas	2511	58.12%	162.9	888.59
max_replicas	2149	81.32%	147.5	275.36
Moldable	2111	71.54%	92.93	273.89

Moldable Scheduling

- Resource adaptivity in job scheduler alone isn't sufficient to maximize cluster utilization
- Need elasticity support from the programming model for efficient HPC on cloud
- The Charm++ programming model provides this elasticity

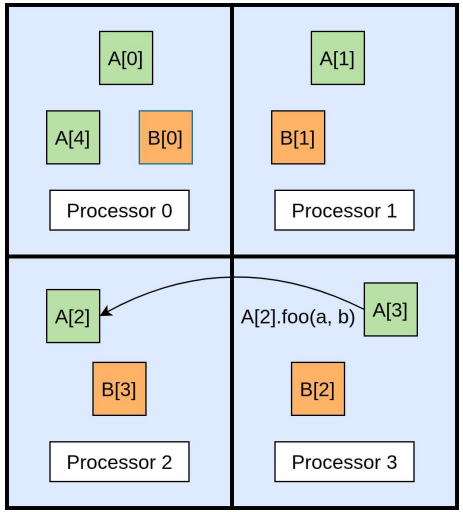


Objectives

- Develop an elastic job scheduler for HPC applications on cloud resources
- Maximize cluster utilization and reduce end-to-end runtime
- Without impacting completion times of high-priority jobs

Background

- Charm++ is an asynchronous message-driven parallel programming model
- Users express computation in terms of objects (chares) that communicate via remote method invocations
- Features -
  - Collections of chares
  - Distributed location management
  - Collectives over chare collections
  - Dynamic load balancing



Why Charm++?

- **Migratable objects**
  - Chares are migratable and can be moved between processors by the runtime system
  - Enables dynamic load balancing

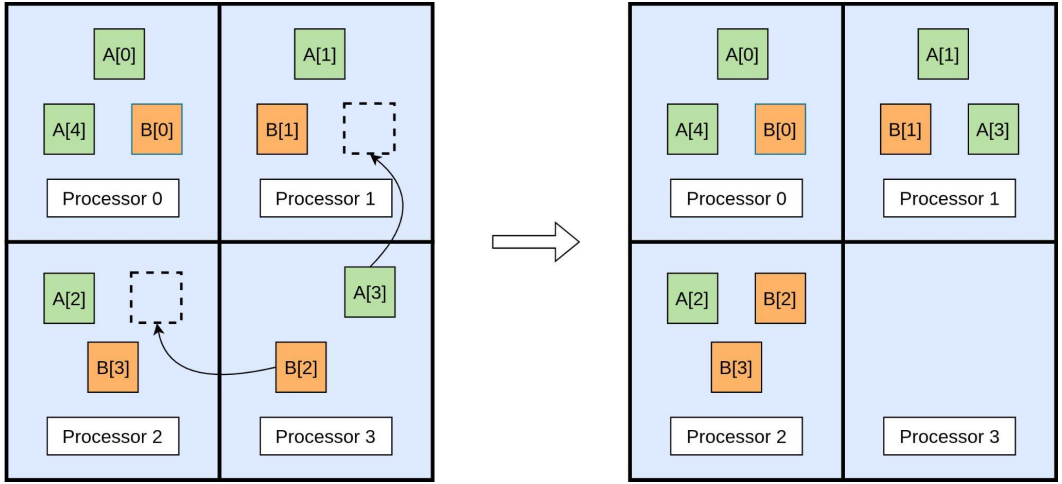
- **Native Elasticity**
  - Migratability allows runtime to natively support elasticity

  - **Shrink** - Move chares off PEs
  - **Expand** - Load balancing chares onto new PEs

  - Can be triggered by an external program

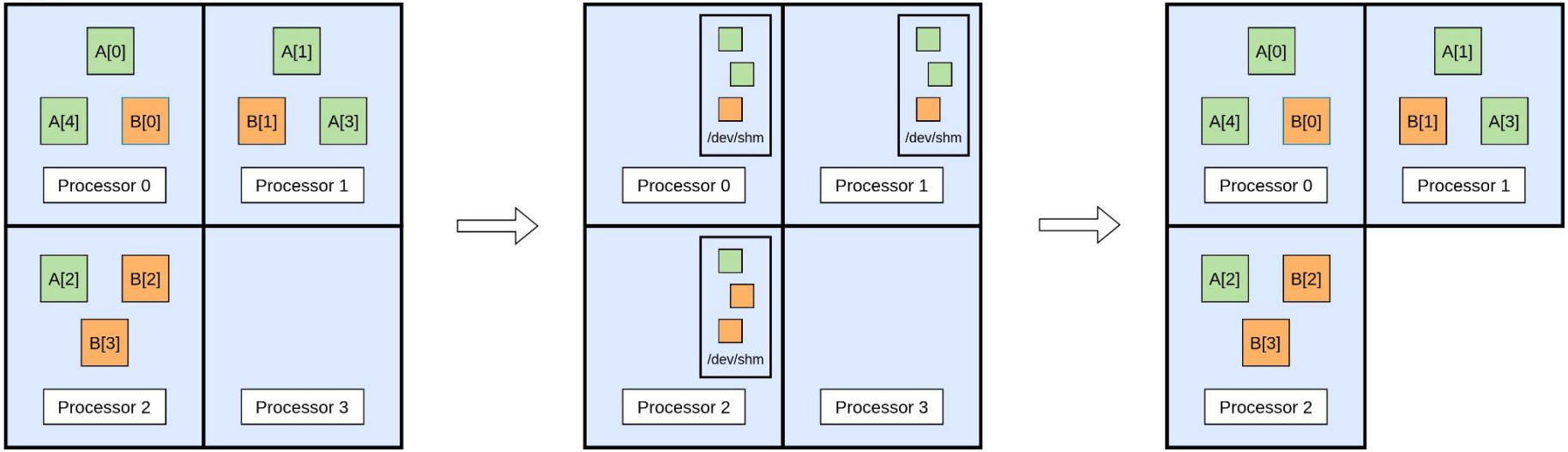
- With minimal extra programmer effort

Elasticity in Charm++



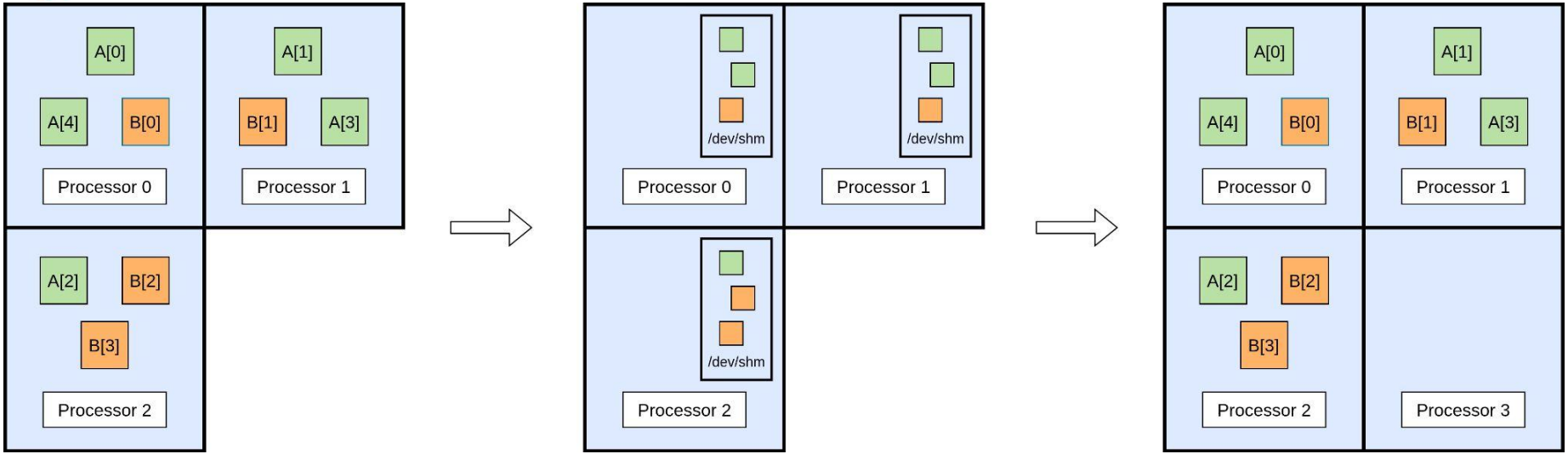
Shrinking from 4 to 3 processors

Elasticity in Charm++



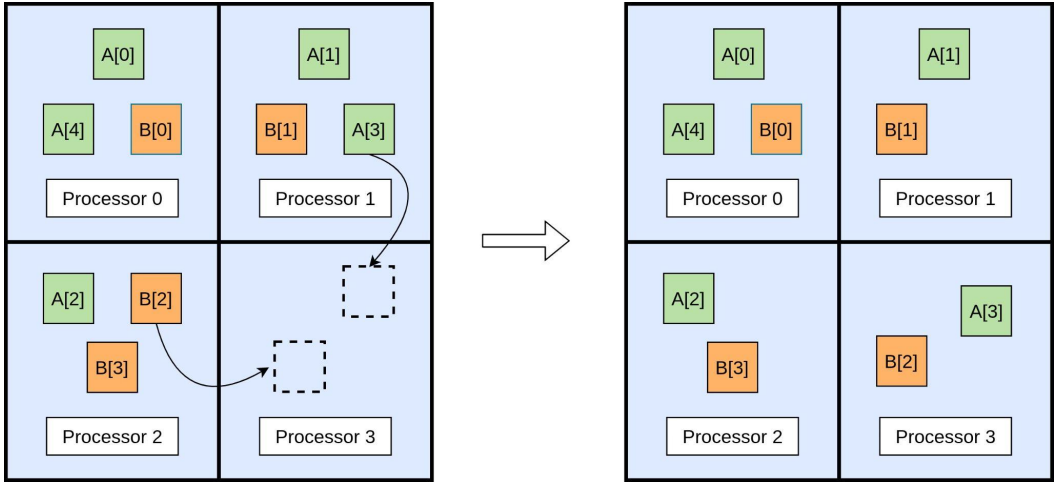
Shrinking from 4 to 3 processors

Elasticity in Charm++



Expanding from 3 to 4 processors

Elasticity in Charm++



Expanding from 3 to 4 processors

Charm++ on Kubernetes

- Modified MPI operator CRD to include
  - `priority`
    Job priority
  - `minReplicas, maxReplicas`
    Minimum and maximum number of worker replicas for a job
- Modified controller for elastic scheduling

Elastic Scheduler Policy (Part 1): New Job Arrives

- **When a new job arrives**

1. **Check Free Slots**: Can the job run (atleast `minReplicas`)? If yes, launch it
2. **No Free Slots? Make room**: Identify lowest priority running jobs
3. **Shrink**: Send *shrink* command to these low priority jobs (down to `minReplicas`) one by one
4. **Launch**: Once enough resources are freed, the new high-priority job is launched.

Elastic Scheduler Policy (Part 2): Job Completes

- **When any job completes**

1. **Find Free Slots**: A number of slots are now free.

2. **Check Priorities**: The scheduler looks at all running and queued jobs, starting with the highest-priority ones.

3. **Expand**: It finds the highest-priority job that is running below its `maxReplicas` and *expands* it to fill the free space.

4. **Repeat**: This process repeats until all free slots are filled, maximizing cluster utilization.

Experimental Results

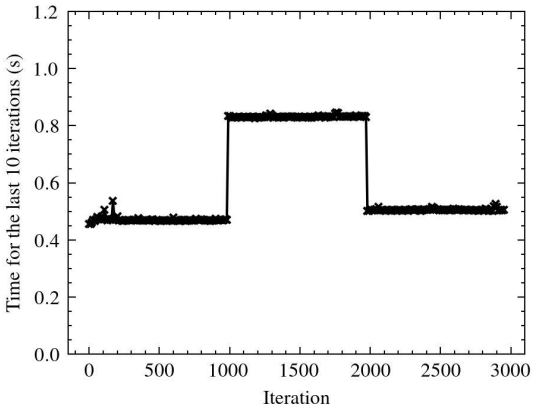
Performance with rescaling

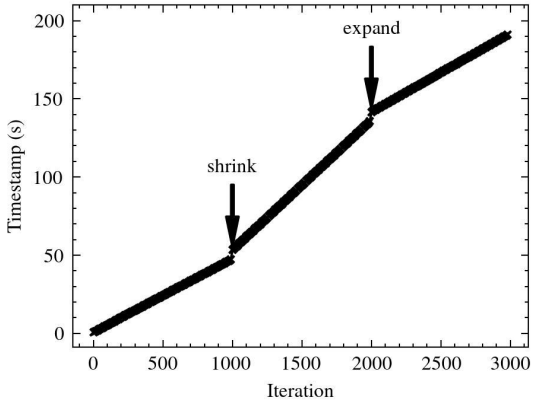
Using the Laplace equation solver application

We start with 32 replicas

Shrink to 16 replicas

Expand back to 32 replicas





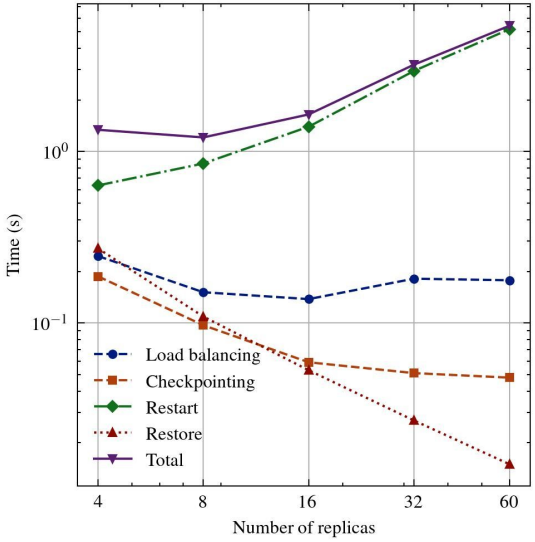
Experimental Results

Overhead of rescaling

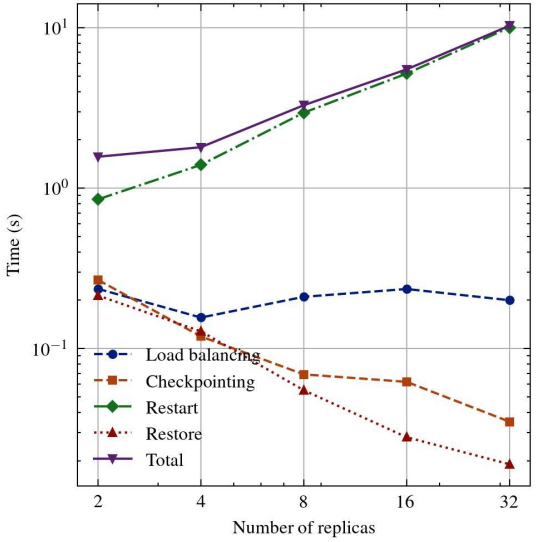
Rescaling overhead has 4 components -

1. Load balancing
2. Checkpointing
3. Restore
4. Restart

Overhead is dominated by restart time



Overhead when shrinking to half the number of replicas



Overhead when expanding to double the number of replicas

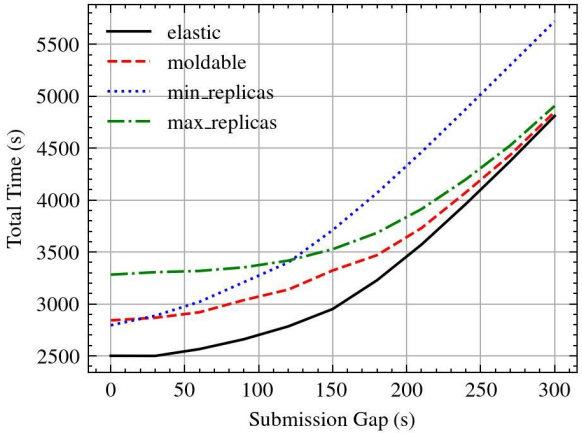
Experimental Results

Elastic Job Scheduler Performance

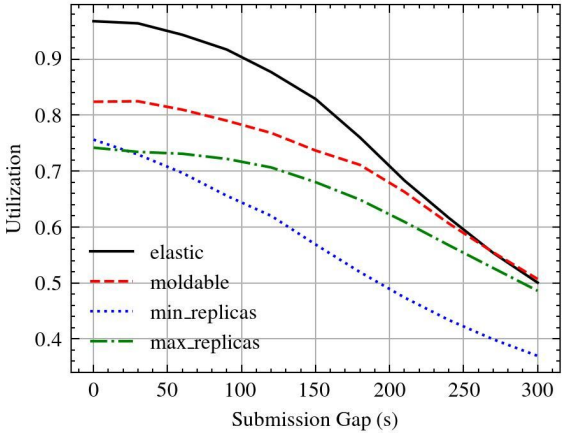
- The job scheduler performance depends on several factors - job sizes, order of submission, gap between submissions
- Infeasible to test all possible configurations
- Simulation of scheduling strategies based on scaling measurements with varying parameters
- Generate a set of jobs with random distribution of job sizes and report average across 100 runs

Experimental Results

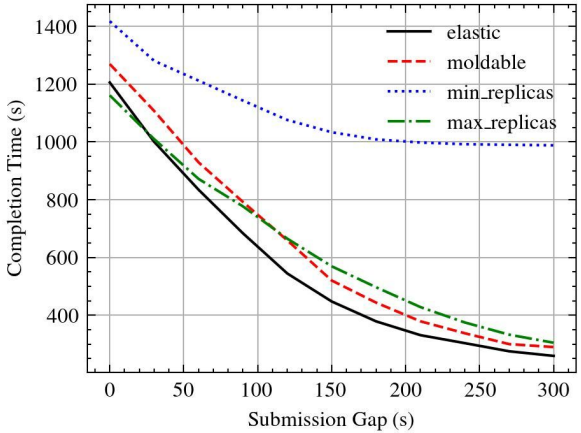
Simulation results with varying submission gap



Total end to end time



Cluster utilization



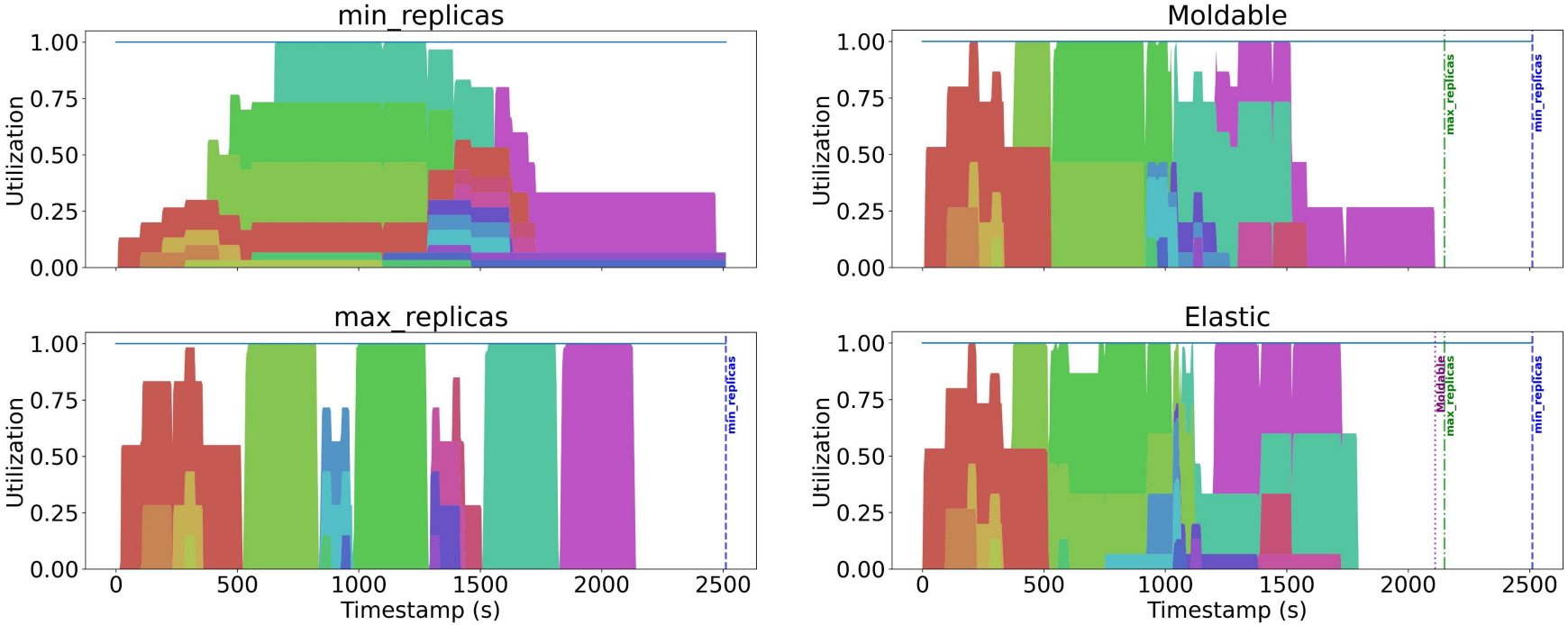
Weighted mean completion time

Experimental Results

Results of run on a Kubernetes cluster

	Total time (s)	Utilization	MRT (s)	MCT (s)
min_replicas	2511	58.12%	162.9	888.59
max_replicas	2149	81.32%	147.5	275.36
Moldable	2111	71.54%	92.93	273.89
Elastic	1796	87.8%	69.93	265.76

Experimental Results



Conclusions

- Effective utilization of cloud resources requires adaptivity-centric design of both the job scheduler and the parallel programming model
- Charm++ provides a programming model with native elasticity support
- The elastic scheduling strategy consistently outperforms rigid and moldable strategies on all metrics

Thank You!